Module 03d — OOP Part 4: Abstract Classes and Interfaces

Phase: Advanced Python
Previous: OOP Part 3: Polymorphism and Project
Next: Exception Handling

Series: OOP overview · Part 1 · Part 2 · Part 3 · Part 4 (this file)

Lab kit: 15_Lab_Assets/32_campus_wallet_abc/

Module Outcome

Lock incomplete blueprints so callers cannot construct them, force children to fill the holes, and treat a methods-only ABC as an interface.

By the end of this module you will be able to:

- Declare an abstract method with `@abstractmethod`
- Build an abstract class (`ABC` + at least one abstract method) and know when `Class()` is illegal
- Treat an ABC with **only** abstract methods as an interface
- Choose interface vs abstract class vs concrete class
- Complete the Campus Wallet kit (savings + payable wallet)

Part 3 already taught duck typing. ABC adds a **construction-time** contract. Exception Handling is **next** — still validate with print + return here.

1. Why This Concept Matters

Duck typing fails when you **call** a missing method. Until then, an incomplete "account" can sit in a list and look fine.

A campus desk wants one slip loop for savings and wallet — and **no** generic `Account("Campus", 0)`. The parent names `kind()`; each child writes the body. Wallet can **pay**; savings must not fake `pay`.

That is an **abstract class** (shared deposit, hole for `kind()`) plus an **interface** (only `pay`). Python has no `interface` keyword — we simulate it with `ABC`.

2. Abstract method

Understand

Sometimes we don't know the implementation yet, but we still want to declare the method. An **abstract method** has a **declaration**, not the real code.

The parent says "every child must have this method." The child writes the body.

`@abstractmethod` lives in the `abc` module. Forget the import → `NameError`. `pass` means "no algorithm yet."

How It Works

```
from abc import ABC, abstractmethod
```

```
class JobRole(ABC):
    @abstractmethod
    def net_pay(self):
        pass

class Intern(JobRole):
    def net_pay(self):
        return 15000

print(Intern().net_pay())
```

Expected Output:

```
15000
```

Abstract method = “I know the name. I do not know the body yet.”

⚠️ WATCH OUT

```
# Wrong — decorator used, abc not imported
class Test:
    @abstractmethod
    def m1(self):
        pass
# NameError: name 'abstractmethod' is not defined
```

Try It

Write abstract `Account` with `kind()`. Implement `SavingsAccount` and `WalletAccount`. Print both kinds. Do not construct the parent.

---

3. Abstract class (`ABC`)

Understand

A **partially implemented** class is an **abstract class**. Share the parts we already know; force children to finish the missing part.

Every abstract class in Python should extend `ABC`. Instantiation depends on **two** things together: did we extend `ABC`, **and** is there at least one `@abstractmethod`?

An abstract class can contain **both** abstract and normal methods. Normal methods are the shared implementation. Abstract methods are the holes.

A child that skips the abstract method is **also abstract**. The `class` line is legal. `Child()` is not.

Can an abstract class have a constructor? **Yes.** `__init__` runs when we construct the **child**, via `super()`. We never construct the abstract parent.

How It Works

Case	Extends `ABC` ?	Has `@abstractmethod` ?	Can you `Test()` ?
1. Normal class	No	No	Yes

Case	Extends `ABC`?	Has `@abstractmethod`?	Can you `Test()`?
2. `ABC` only	Yes	No	Yes — still complete
3. Both	Yes	Yes	**No** — `TypeError`
4. Decorator only	No	Yes	Yes — decorator is documentation
5. Decorator + body, no `ABC`	No	Yes	Yes — runs like a normal method

Conclusion: at least one abstract method **and** `ABC` → instantiation is not possible.

Case 3 — both, blocked:

```
from abc import ABC, abstractmethod

class Test(ABC):
    @abstractmethod
    def m1(self):
        pass

# t = Test()   # TypeError: Can't instantiate abstract class Test ... m1
print("Test class exists. No object — as designed.")
```

Expected Output:

```
Test class exists. No object — as designed.
```

(The exact `TypeError` wording changes slightly by Python version.)

Incomplete child — class line OK, object not:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def no_of_wheels(self):
        pass

class Bus(Vehicle):
    pass

print("Bus class exists. No object yet — no error.")
# b = Bus()  # TypeError — Bus did not override no_of_wheels
```

Expected Output:

```
Bus class exists. No object yet — no error.
```

Children that **do** implement:

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
    @abstractmethod
    def no_of_wheels(self):
        pass

class Bus(Vehicle):
    def no_of_wheels(self):
        return 6

class Auto(Vehicle):
    def no_of_wheels(self):
        return 3

print(Bus().no_of_wheels())
print(Auto().no_of_wheels())
```

Expected Output:

```
6
3
```

Shared methods + a hole — the everyday abstract class:

```
from abc import ABC, abstractmethod

class Account(ABC):
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return
        self._balance += amount

    def get_balance(self):
        return self._balance

    @abstractmethod
    def kind(self):
        pass

class SavingsAccount(Account):
    def kind(self):
        return "savings"

acct = SavingsAccount("Priya", 1000)
acct.deposit(200)
print(acct.kind(), acct.holder, int(acct.get_balance()))
# Account("Campus", 0)  # TypeError — Case 3
```

Expected Output:

```
savings Priya 1200
```

⚠ WATCH OUT

```
# Wrong — class Account(ABC) with no @abstractmethod
# Account("Campus") still works  (Case 2)

# Wrong — @abstractmethod without ABC
# Test() still works             (Case 4)
```

`ABC` alone does not block objects. `@abstractmethod` alone does not block objects. **Both together** do.

Try It

Write abstract `Report` with shared `header()` and abstract `title()`. Implement `LabReport` and `InvoiceReport`. Print header plus title for each. Leave the parent unconstructed.

---

4. Interface in Python

Understand

A **wall socket**: any appliance with the same pins works. The socket does not know *how* the appliance uses power.

If an abstract class contains **only abstract methods**, we consider it an **interface**. Python has no `interface` keyword.

Write business code against the contract. Swap the real class later without rewriting the caller.

`typing.Protocol` describes an interface **without** inheriting `ABC` — structural ("if it has `pay`, it counts"), mainly for type checkers. Labs use `ABC`. Duck typing (Part 3) fails when we **call** a missing method. ABC fails when we **construct** an incomplete child. Named contracts also let us write `isinstance(x, Payable)`.

How It Works

```
from abc import ABC, abstractmethod

class DBInterface(ABC):
    @abstractmethod
    def connect(self):
        pass

    @abstractmethod
    def disconnect(self):
        pass

class MySQL(DBInterface):
    def connect(self):
        print("Connecting to MySQL...")

    def disconnect(self):
        print("Disconnecting MySQL...")

class Postgres(DBInterface):
    def connect(self):
        print("Connecting to Postgres...")

    def disconnect(self):
        print("Disconnecting Postgres...")

engines = {"MySQL": MySQL, "Postgres": Postgres}
```

```
db = engines["MySQL"]()
db.connect()
db.disconnect()
```

Expected Output:

```
Connecting to MySQL...
Disconnecting MySQL...
```

Change the key to `"Postgres"` — the same `connect()` / `disconnect()` calls talk to a different engine.

A class may take **two** interface-style parents: `class WalletAccount(Account, Payable)`. Savings **is an** account. A wallet **can** pay. Do not put `pay` on `Account`, or every savings account is forced to fake a payment.

Keep state (`holder`, `_balance`) on abstract `Account`. Keep `Payable` as methods only.

⚠️ WATCH OUT

```
# Wrong
DBInterface()           # Case 3 — construct MySQL() or Postgres()

# Wrong
class Payable(ABC):
    def __init__(self, balance): ...   # state does not belong on the interface
```

Try It

Write a `Notifier` interface (`ABC` + only `send`). Implement `Sms` and `Email`. Loop both to send `"Order shipped"`.

---

5. Concrete vs abstract vs interface

Understand

Three documents on a campus project:

1. Know **only** the requirement → **interface**.
2. Some real code, some holes → **abstract class**.
3. Implementation complete → **concrete class**.

	Interface	Abstract class	Concrete class
What we know	Only method names	Some real code + some holes	Full service
Python	`ABC` + **only** `@abstractmethod`	`ABC` + mix of real and abstract methods	Normal class, or child that filled every hole
Construct?	No	No	Yes
Example	`Payable`, `DBInterface`	`Account` with `deposit` + abstract `kind`	`SavingsAccount`, `MySQL`

The three layers can sit in a chain. Only the bottom class is constructible.

Why not put `pay` on abstract `Account`? Not every account pays. `Payable` is the requirement for types that **can** pay.

Do not call `ABC` "Python's interface keyword." `ABC` is the abstract-class tool. Using it with **only** abstract methods is how we **simulate** an interface.

How It Works

```
from abc import ABC, abstractmethod

class CollegeAutomation(ABC):
    @abstractmethod
    def m1(self):
        pass

    @abstractmethod
    def m2(self):
        pass

    @abstractmethod
    def m3(self):
        pass

class AbsCls(CollegeAutomation):
    def m1(self):
        print("m1 method implementation")

    def m2(self):
        print("m2 method implementation")

class ConcreteCls(AbsCls):
    def m3(self):
        print("m3 method implementation")

c = ConcreteCls()
c.m1()
c.m2()
c.m3()
```

Expected Output:

```
m1 method implementation
m2 method implementation
m3 method implementation
```

`AbsCls()` would still be illegal — `m3` is missing.

Keep `pay` off `Account`. Loop and call `pay` only when `isinstance(a, Payable)`:

```
from abc import ABC, abstractmethod

class Account(ABC):
    def __init__(self, holder):
        self.holder = holder

    @abstractmethod
    def kind(self):
        pass
```

```
class Payable(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class SavingsAccount(Account):
    def kind(self):
        return "savings"

class WalletAccount(Account, Payable):
    def kind(self):
        return "wallet"

    def pay(self, amount):
        print("pay ₹" + str(int(amount)), "from", self.holder)
        return True

accounts = [SavingsAccount("Priya"), WalletAccount("Ravi")]
for a in accounts:
    print(a.kind(), a.holder)
    if isinstance(a, Payable):
        a.pay(80)
```

Expected Output:

```
savings Priya
wallet Ravi
pay ₹80 from Ravi
```

Do not write `if a.kind() == "wallet"` to call `pay`.

⚠ WATCH OUT

Putting `pay` on `Account` forces every child to implement it — even types that never pay a merchant. Split the contract.

Try It

Make `CampusWallet` implement both `Printable` (`print_slip`) and `Payable` (`pay`). Print a slip, pay ₹120 from ₹500, print the slip again. Rejecting a bad pay with print + return is enough.

Step-by-Step: Build It

One assemble: an abstract `Account` with shared money methods and a `kind()` hole, plus two concrete children. This is the **pattern** for Campus Wallet. The kit still needs `withdraw`, `Payable.pay`, and `add_interest` — do not copy a finished wallet from here.

```
from abc import ABC, abstractmethod

class Account(ABC):
    def __init__(self, holder, opening):
        self.holder = holder
        self._balance = float(opening)

    def deposit(self, amount):
        if amount <= 0:
            print("amount must be positive")
```

```
            return
        self._balance += amount

    def get_balance(self):
        return self._balance

    @abstractmethod
    def kind(self):
        pass

    def print_slip(self):
        print(self.kind(), "|", self.holder, "₹" + str(int(self._balance)))

class SavingsAccount(Account):
    def kind(self):
        return "savings"

class WalletAccount(Account):
    def kind(self):
        return "wallet"

accounts = [SavingsAccount("Priya", 1000), WalletAccount("Ravi", 500)]
for a in accounts:
    a.print_slip()

accounts[0].deposit(40)
accounts[0].print_slip()
print("Ravi:", int(accounts[1].get_balance()))
# Account("Campus", 0)  # TypeError — parent is abstract
```

Expected Output:

```
savings | Priya ₹1000
wallet | Ravi ₹500
savings | Priya ₹1040
Ravi: 500
```

`print_slip` is a **normal** method on `Account` — that is why `Account` is an abstract class, not an interface. On the kit, add `withdraw`, make wallet also `Payable`, and implement `pay` **using** `withdraw` (print + return).

Practice Exercises

Attempt each problem before reading the model answer.

1. What is an abstract method in one or two sentences? Why do we import from `abc`?

Model answer:
An abstract method is declared even though we do not know the implementation yet — in Python: `@abstractmethod` with `pass`. `@abstractmethod` and `ABC` live in the `abc` module; skip the import → `NameError`.

2. Write the conclusion of Cases 1–5 in one line. `class Bus(Vehicle): pass` when `Vehicle` is `ABC` with abstract `no_of_wheels` — is the `class` line legal? Is `Bus()` legal?

Model answer:
If a class contains at least one abstract method **and** extends `ABC`, instantiation is not possible. The `class Bus` line is legal; `Bus()` is not — `Bus` did not override `no_of_wheels`, so it is still abstract (`TypeError`).

3. Create abstract `Shape` with `area()`, `Circle(r)` using `3.14 * r * r`, `Rect(w, h)` as `w * h`. Print both areas as integers.

Model answer:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14 * self.r * self.r

class Rect(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h

print(int(Circle(2).area()))
print(int(Rect(3, 4).area()))
```

Expected Output:

```
12
12
```

4. When do we pick interface vs abstract class vs concrete class?

Model answer:
Interface: we know only the requirement (`connect` / `pay`). Abstract class: we have some real code and some holes (`deposit` written, `kind` missing). Concrete class: implementation is complete and we can create objects (`MySQL()`, `SavingsAccount(...)`).

5. Sketch a tiny `Payable` interface (ABC with only `pay`) and one concrete class that implements it. Call `pay` through a list.

Hint: Loop `for p in [Wallet(...)]: p.pay(50)`.

Model answer:

```
from abc import ABC, abstractmethod

class Payable(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class Wallet(Payable):
    def pay(self, amount):
```

```
        print("paid", amount)
        return True

for p in [Wallet()]:
    p.pay(50)
```

Expected Output:

```
paid 50
```

6. Why not put `pay` on abstract `Account`? Answer in 2–3 sentences.

Model answer:
Not every account pays a merchant. `Payable` is a separate requirement. Wallet opts in; savings does not fake `pay`. Call `pay` when `isinstance(a, Payable)`, not when `kind() == "wallet"`.

Lab

Combine ABC, an abstract method, a normal shared method, and an interface. Not the Campus Wallet kit.

Problem

A drawing tool needs shapes that can report `area()`. You must not construct a generic `Shape`. A separate `Drawable` contract only requires `draw()`.

What to Build

Abstract `Shape` with abstract `area()` and a normal `label()` that prints the class kind plus area. Interface `Drawable` with only `draw()`. `Circle` implements both. A short demo.

Requirements

- `Shape(ABC)` — abstract `area()`; `label()` prints a one-line summary
- `Drawable(ABC)` — only abstract `draw()`
- `Circle(Shape, Drawable)` — `area` uses `3.14 * r * r`; `draw` prints `"circle r=..."`
- Demo: construct `Circle(2)`, print `label()`, call `draw()`, print `int(area())`
- Leave `Shape(2)` unconstructed (would be `TypeError`)

Expected Behavior

`Circle` works. A bare `Shape` is not allowed. `draw` is not on `Shape`.

Sample Run

```
circle area 12
circle r=2
12
```

(Your wording may differ; the **rules** must match.)

Constraints

No `raise` / `try` / `except` in your code. No files. No type hints required.

Bonus

Add `Rect(w, h)` as `Shape` only (not `Drawable`). Loop shapes for `label()`; call `draw` only if `isinstance(s, Drawable)`.

Mini-Project (Assignment): HDFC Bank Campus Wallet

Why this matters: One slip loop, two account kinds, and pay as an **opt-in** contract — the same split you will see in payment and plugin APIs.

Lab kit: 15_Lab_Assets/32_campus_wallet_abc/ — complete `starter/`. Do **not** copy `solution/`.

This is **not** the Shape lab. New kit. Finish `kind()`, `add_interest`, and `WalletAccount.pay`.

Objective

Savings and wallet share an abstract `Account`. Wallet also implements `Payable`. A generic `Account(...)` must not be allowed.

Story / Scenario

HDFC Bank campus desk prints one slip loop. Wallet can **pay**. Savings adds interest. Validation stays print + return.

Requirements

Piece	Detail
`Account(ABC)`	Abstract class: holder, balance, `deposit` / `withdraw` / `get_balance` / `print_slip`; abstract `kind()`
`Payable`	Interface: only `pay(amount)`
`SavingsAccount`	Concrete; `kind()` → `"savings"`; `add_interest()`
`WalletAccount`	Concrete; `Account` **and** `Payable`; `pay` uses `withdraw` (print + return)
Demo	List of accounts; print slips; pay from wallet; add interest on savings

Do not write `if a.kind() == "wallet"` to call `pay`. Use `isinstance(a, Payable)` or call `pay` on the wallet object.

Expected Behavior

- `Account("Campus", 0)` is not allowed
- Both kinds print a slip through the parent type
- Wallet `pay` succeeds and fails (print + return) without `raise`
- Savings can `add_interest()`; it does not implement `pay`

Sample Run

```
savings | Priya ₹1000
wallet | Ravi ₹500
Paid ₹120 from wallet
pay ok? True
wallet | Ravi ₹380
```

```
savings | Priya ₹1040
insufficient funds
rejected? False
```

(Kit print wording may differ slightly; the **rules** must match.)

Bonus Challenges

- Reject a zero / negative pay with a print; keep the old balance
- Add `CurrentAccount` that is an `Account` but **not** `Payable`
- Tiny CLI around the list (Deposit / Withdraw / Pay / Interest / List / Quit)
- **Later — Exceptions week:** `raise` on bad amounts; catch in the menu

Done when

- [ ] `Account("Campus", 0)` is not allowed
- [ ] Savings and wallet both print a slip through the parent type
- [ ] Wallet `pay` uses `withdraw` (print + return) without `raise`
- [ ] Savings can `add_interest()`; it does not implement `pay`
- [ ] No type hints required

Interview Q&A

Q1. What is an abstract method?
A method with declaration only, no real implementation. Python: `@abstractmethod` and `pass`. Child classes write the body.

Q2. What is an abstract class? Can we instantiate it?
A partially implemented class. Extend `ABC` and include at least one abstract method. Then we cannot create an object (`TypeError`). Construct the concrete child instead.

Q3. Extending `ABC` with no abstract method — can we create an object?
Yes (Case 2). Both `ABC` **and** an abstract method are required to block instantiation.

Q4. Child does not override the abstract method. What happens?
The child is also abstract. The `class` line is fine. Creating the child object raises `TypeError`.

Q5. What is an interface in Python?
There is no `interface` keyword. An abstract class with **only** abstract methods is the interface. Example: `DBInterface` with `connect` / `disconnect`.

Q6. When interface, when abstract class, when concrete class?
Interface = only the requirement. Abstract class = partial implementation. Concrete class = complete, ready to create objects.

Q7. Why not put `pay` on `Account`?
Not every account pays a merchant. `Payable` is a separate requirement. Wallet opts in; savings does not fake `pay`.

Q8. Can an abstract class contain normal methods?
Yes. That is the difference from an interface. `deposit` / `print_slip` on `Account` are normal; only `kind()` is abstract.

Q9. `ABC` vs `Protocol`?
`ABC` — inherit the contract; missing methods fail at construction. `Protocol` — structural ("if it has `pay`, it counts"); mainly for type checkers. Labs use `ABC`.

Q10. Duck typing vs ABC — what extra do we get from `@abstractmethod`?
Duck typing fails when we **call** a missing method. An ABC fails when we **construct** an incomplete child. Named contracts also let us write `isinstance(x, Payable)`.

Q11. Why give an abstract class `__init__`?
Children still need holder and opening balance. `__init__` runs through `super()` when we construct the child — never as `Account(...)`.

Cheat Sheet

```
from abc import ABC, abstractmethod
@abstractmethod + pass     → hole (no body yet)
ABC + abstract method      → cannot construct (TypeError)
ABC alone                  → objects still allowed (Case 2)
@abstractmethod alone      → objects still allowed (Case 4)
child skips the method     → child is still abstract
interface                  → ABC with only abstract methods
abstract class             → mix of real methods + holes
concrete class             → every hole filled; construct OK
Protocol                   → structural interface; labs use ABC
isinstance(x, Payable)     → opt-in contract; not if kind == ...
```

Validate with print + return. Exception Handling is next.

Completion Checklist

```
[ ] I can declare @abstractmethod and import from abc
[ ] I know ABC + abstract method together block Class()
[ ] I know a child that skips the method is still abstract
[ ] I can tell interface vs abstract class vs concrete class
[ ] I keep pay off Account and use Payable for types that pay
[ ] I completed the lab
[ ] I attempted the Campus Wallet kit
[ ] I can explain ABC vs duck typing vs Protocol in an interview
```

What's Next?

You can now lock incomplete blueprints and swap implementations behind a contract.

Finish the Campus Wallet lab kit, then continue with **Exception Handling** — that is where `try` / `except` / `raise` belong, including safer CLI input.